j4_controller

Firmware for the **Johnny 4 robot controller/transmitter**. Runs on a LILYGO TTGO T-Display v1.1 (ESP32). Reads potentiometers and two 4x4 keypads, ingests four more pots from the j4_display_right board, transmits control data to the robot receiver over ESP-NOW, and mirrors live data to the two XIAO display boards over UART.

What is Johnny 4?

Johnny 4 is a prop robot controlled wirelessly. This board is the handheld controller that the operator uses to drive the robot's servos, select audio phrases, and monitor battery status.

What this firmware does

- Reads analog potentiometers via four local ADS1115 ADC modules: the eyes joystick (eyes_x / eyes_y), the neck joystick (neck X / jaw Y), and the eight middle face pots (Eyebrow L/R, Basket Eyebrow L/R, Nose, Nose Basket, Bottom Eyelid L/R); the fourth module carries the two linear faders, the Nose Basket pot, and the disabled NECK PIV pot
- Reads the four panel toggle switches (LASER, VENT, EYE POP, and the AUX toggle by the right joystick) on GPIOs 32/33/13/15 with internal pull-ups
- Ingests four more pots (iris, color, brightness, volume) from j4_display_right's dedicated ADS1115 over Serial2 as 25 Hz `P:` lines
- Reads two 4x4 matrix keypads via PCF8574 I2C expanders using a custom scan routine
- Left keypad: jukebox-style audio phrase selection: press a letter (A-D) then a digit (0-9) to queue a phrase, or press a digit alone to queue a 0x phrase (e.g. pressing 8 queues "08.wav")
- Right keypad: face presets. Tap a key to recall its saved face; hold a key 3 seconds to save the current face to it (j4_display_right shows "FACE SAVED TO `<key>`"). Faces live on the microSD on j4_display_right's TFT module, with a cache in this board's NVS flash so saves are immediate and recall works with that board unplugged
- Press `*` to send a STOP command, press `#` to clear the buffer
- Mixes the joystick into differential neck-left / neck-right values, both centre-zero (**-1600..0..1600**). Both axes are sticky-filtered at the source so a parked (spring-removed) stick holds dead still without giving up 1-count precision when moved slowly
- Transmits all control data to the robot receiver via ESP-NOW (fixed-size packed structs, no `String` members so they survive the wireless `memcpy`)
- Receives the jukebox file list from the robot receiver in chunks and forwards it to the display board; carries a `need_filelist` flag so the receiver re-sends the list if this board boots late
- Answers `LIST?` requests from the display board out of its cached copy, so a display reboot recovers the list without a full round trip
- Displays live pot values, keypress state, and battery voltage on the built-in TFT. The default (data) screen reads: Keypad_L, Playing, VOL, Keypad_R, Eye-X, Eye-Y, Neck-L, Neck-R, Neck-PIV, then the STATUS line. Each keypad has its own last-key line
- The TTGO's built-in button (GPIO 35) cycles the TFT through eight screens: live data, WiFi MAC address, a connection-status screen showing ESP-NOW LINK, j4_stepper_neck, j4_stepper_eyes, j4_talk, j4_display_left, and j4_display_right as CONNECTED (green) / DISCONNECTED (red), then five more, one per ADS1115 in the system: ADS_01 through ADS_04 on this board, plus the fifth on j4_display_right (IRIS, COLOR, BRIGHTNESS, VOLUME) fed by its `P:` lines rather than by I2C. Each shows eight values: every channel's live **raw** count, and under it what that channel drives plus the processed value that leaves this board. Raw is what you need to calibrate a fader window or a joystick centre, so this replaces putting a laptop on the I2C bus
- Sends a 49-byte binary status packet to the j4_display_left board at 25 fps over UART
- Shows a STATUS line on the built-in TFT: "ONLINE" when the ESP-NOW link is up and the steppers are healthy, "OFFLINE" if no status packet arrives, or the reported stepper fault (e.g. "NL OT", "EYES OFFLINE"); green when healthy, red on any fault

Hardware

Component	Details
Microcontroller	LILYGO TTGO T-Display v1.1 (ESP32 with built-in 1.14" ST7789 TFT)
ADC modules	Four ADS1115 (I2C addresses 0x48, 0x49, 0x4A, 0x4B); a fifth lives on j4_display_right
Toggle switches	LASER, VENT, EYE POP, AUX -- GPIOs 32/33/13/15, INPUT_PULLUP, switch closes to GND
Keypad expanders	Two PCF8574 I2C I/O expanders: 0x21 (keypad_left, A0 jumper high) and 0x20 (keypad_right)
Keypads	Two 4x4 matrix keypads, headers soldered directly into P0-P7; left = phrase select, right = face presets
Displays	j4_display_left (jukebox, Serial1) + j4_display_right (pot labels, Serial2)

Pin assignments

GPIO	Function
21	I2C SDA (ADS1115 x4, PCF8574 keypads x2)
22	I2C SCL
17	Serial1 TX to j4_display_left (XIAO D7)
27	Serial1 RX from j4_display_left (XIAO D6)
25	Serial2 TX to j4_display_right (XIAO D7: face values, messages, face store)
26	Serial2 RX from j4_display_right (XIAO D6, pot feed)
32	LASER toggle (INPUT_PULLUP, switch closes to GND)
33	VENT toggle (INPUT_PULLUP, switch closes to GND)
13	EYE POP toggle (INPUT_PULLUP, switch closes to GND)
15	AUX toggle, next to the right joystick (INPUT_PULLUP, unassigned)
34	Battery voltage sense (ADC input, read-only)
35	Screen-cycle button (TTGO built-in; data / MAC / connection status)

GPIO 35 is the TTGO's on-board button, so it needs no wiring. Potentiometer inputs are routed through the ADS1115 modules. GPIO 12 causes boot/flash issues when connected to a pot and is avoided. GPIO 17 is unreliable as an input and is used only as TX. Pins 36, 37, and 38 are input-only. The four toggles use the internal pull-ups, so each switch just shorts its GPIO to GND when flipped ON -- no external resistors.

Pin diagram

Physical layout, display facing AWAY from you (rails visible), USB-C at the TOP (matching how the board is mounted). The two header rails read top-to-bottom as shown.

```
           LEFT rail                              RIGHT rail
                                +--[ USB-C ]--+
                    3.3V -- 3V3|             |  5V
                  ground -- GND|             |  GND  ground
                  ground -- GND|     TTGO    |  27   DISP-L RX <- XIAO D6
     (avoid: boot/flash) -- 12 |  T-Display  |  26   DISP-R RX <- XIAO D6
          EYE POP toggle -- 13 |     v1.1    |  25   DISP-R TX (values+msgs)
              AUX toggle -- 15 |             |  33   VENT toggle
                  (free) -- 2  |   [ ST7789  |  32   LASER toggle
    DISP-L TX -> XIAO D7 -- 17 |    TFT on   |  39   (input only)
                 I2C SCL -- 22 |    back ]   |  38   (input only)
                 I2C SDA -- 21 |             |  37   (input only)
                  ground -- GND|             |  36   (input only)
                  ground -- GND|             |  3.3V
                                +-------------+

   on-board (no header pin): GPIO34 = battery sense, BOOT = GPIO0,
       GPIO35 = screen-cycle button, TFT on 4/5/16/18/19/23
     I2C bus: ADS_01 @ 0x48, ADS_02 @ 0x49, ADS_03 @ 0x4A, ADS_04 @ 0x4B,
       PCF8574 keypad_left @ 0x21, keypad_right @ 0x20
     toggles: INPUT_PULLUP, switch closes to GND (ON = LOW)
```

Corrected against LilyGO's own pinout diagram for the T-Display v1.1 (24 pins, 12 per rail, drawn from the front with the display facing the viewer and USB-C at the bottom). Getting from that diagram to this one takes two transforms, not one: rotate 180 degrees so USB-C ends up at the top (this alone swaps which rail each pin is on *and* reverses top-to-bottom order), then mirror left-right because this diagram is drawn from the back (rails visible, display facing away) rather than the front (a plain left/right swap, order unchanged). Composed, the right rail happens to end up identical to where it started; the left rail does not. The previous version of this diagram had the left rail's pins in the un-rotated order and was missing a GND pin on each rail (11 shown per side against the board's actual 12) -- both are fixed above.

ESP-NOW (wireless) links this board to j4_receiver.

ADS1115 pin diagrams

All four modules share the TTGO's I2C bus (SDA = GPIO 21, SCL = GPIO 22, 3.3V, GND); only the ADDR strap and the four analog channels differ. Every pot's wiper goes to its A-pin; the outer legs go to 3.3V and GND.

```
ADS_01 @ 0x48 (ADDR -> GND)                 both joysticks
+------+
| VDD  |  3.3V
| GND  |  GND
| SCL  |  TTGO GPIO 22 (I2C SCL)
| SDA  |  TTGO GPIO 21 (I2C SDA)
| ADDR |  GND  = address 0x48
| ALRT |  not connected
| A0   |  neck joystick X wiper        -> neck L/R differential mix
| A1   |  neck joystick Y (jaw) wiper  -> neck L/R base height
| A2   |  eyes joystick X wiper        -> eyes pan servo  (PCA9685 ch 3), -128..0..128
| A3   |  eyes joystick Y wiper        -> eyes tilt servo (PCA9685 ch 4), -128..0..128
+------+

ADS_02 @ 0x49 (ADDR -> VDD)                 four face pots, left bank
```

```
+------+
| VDD  |  3.3V
| GND  |  GND
| SCL  |  TTGO GPIO 22 (I2C SCL)
| SDA  |  TTGO GPIO 21 (I2C SDA)
| ADDR |  VDD  = address 0x49
| ALRT |  not connected
| A0   |  Eyebrow L pot wiper          -> PCA9685 ch 6
| A1   |  Eyebrow R pot wiper          -> PCA9685 ch 7
| A2   |  Basket Eyebrow L pot wiper   -> PCA9685 ch 8
| A3   |  Basket Eyebrow R pot wiper   -> PCA9685 ch 9
+------+

ADS_03 @ 0x4A (ADDR -> SDA)                three face pots, right bank
+------+
| VDD  |  3.3V
| GND  |  GND
| SCL  |  TTGO GPIO 22 (I2C SCL)
| SDA  |  TTGO GPIO 21 (I2C SDA)
| ADDR |  SDA  = address 0x4A
| ALRT |  not connected
| A0   |  Nose pot wiper (up/down)     -> PCA9685 ch 10
| A1   |  FAULTY channel, nothing connected, left unread in firmware
| A2   |  Bottom Eyelid L pot wiper    -> PCA9685 ch 12
| A3   |  Bottom Eyelid R pot wiper    -> PCA9685 ch 13
+------+

ADS_04 @ 0x4B (ADDR -> SCL)                neck-pivot + faders + Nose Basket
+------+
| VDD  |  3.3V
| GND  |  GND
| SCL  |  TTGO GPIO 22 (I2C SCL)
| SDA  |  TTGO GPIO 21 (I2C SDA)
| ADDR |  SCL  = address 0x4B
| ALRT |  not connected
| A0   |  NECK PIV pot -- DISABLED, read for display only (to be repurposed)
| A1   |  fader_left wiper             -> neck pivot, sole source (-1600..0..1600)
| A2   |  fader_right wiper            -> iris (doubles j4_display_right IRIS), 0-255
| A3   |  Nose Basket pot wiper        -> PCA9685 ch 11 (single source)
+------+
```

Analog inputs

Local, on the four ADS1115 modules over I2C:

Module	Channel	Control
ADS_01 (0x48)	A0	Neck joystick X
ADS_01 (0x48)	A1	Neck joystick Y (jaw, feeds neck mixer)

ADS_01 (0x48)	A2	Eyes joystick X / eyes_x
ADS_01 (0x48)	A3	Eyes joystick Y / eyes_y
ADS_02 (0x49)	A0	Eyebrow L pot
ADS_02 (0x49)	A1	Eyebrow R pot
ADS_02 (0x49)	A2	Basket Eyebrow L pot (was eye-pop, now a toggle)
ADS_02 (0x49)	A3	Basket Eyebrow R pot
ADS_03 (0x4A)	A0	Nose pot (up/down)
ADS_03 (0x4A)	A1	Faulty channel, unused (Nose Basket moved to ADS_04 A3)
ADS_03 (0x4A)	A2	Bottom Eyelid L pot
ADS_03 (0x4A)	A3	Bottom Eyelid R pot
ADS_04 (0x4B)	A0	NECK PIV pot -- **disabled**, read for display only, awaiting a new use
ADS_04 (0x4B)	A1	fader_left (linear fader) -- sole source for the neck pivot, -1600..0..1600 over the bottom 40% of its travel
ADS_04 (0x4B)	A2	fader_right (linear fader, doubles the IRIS pot), 0-255 over the bottom 40% of its travel
ADS_04 (0x4B)	A3	Nose Basket pot (up/down, single source)

The eight face pots ride the ESP-NOW control packet to PCA9685 channels 6-13 on j4_receiver (Eyebrow L/R = ch 6/7, Basket Eyebrow L/R = ch 8/9, Nose = ch 10, Nose Basket = ch 11, Bottom Eyelid L/R = ch 12/13). The neck-pivot pot rides the packet in its own field and leaves the receiver as the `nP` slot of the stepper stream.

Fader travel windows

Each fader uses a window at the bottom of its mechanical travel:

Fader	raw at low end	raw at centre	raw at high end	past the window
fader_left (neck pivot)	17560 = **-1600**	15000 = **0**	12440 = **+1600**	stays +1600
fader_right (iris)	0 = **0**	3512 = **127**	7024 = **255**	stays 255

fader_left's window spans 5120 raw counts (about 29% of its mechanical travel), positioned so centre lands exactly on raw 15000. fader_right uses the bottom 40%.

fader_left is mounted inverted relative to fader_right, so its physical bottom is the numerically *higher* raw count. That inversion is expressed by the endpoint order rather than a separate flag.

The window is defined by **measured raw endpoints** (`FADER_L_RAW_BOTTOM` / `_TOP` , `FADER_R_RAW_BOTTOM` / `_TOP`), not by percentages of the ADC range. A fader's electrical span reaches neither the ADC rails nor the ends of its own mechanical travel, so the physical-to-raw relationship has to be measured. Assuming it was linear across the full ADC range is what left a stretch of dead motion at the bottom of fader_left's throw.

Calibrated 2026-08-30: fader_left reads 17560 raw at its physical bottom, fader_right reads 0 at its. 17560 counts is 3.29V, so the faders span the full 3.3V rail end to end. fader_left's window was then narrowed by hand to put centre on raw 15000, with equal 2560-count halves either side.

Recalibrating a fader window. Cycle the TFT to the ADS_04 screen, which shows raw counts. Park the fader at its physical bottom and read the FADER L / FADER R value: that is `_RAW_BOTTOM` . Move it to the 40% mark and read it again: that is `_RAW_TOP` . Put both in `main.cpp` and reflash. The 20% (centre) point falls out of the linear map. Getting `_RAW_BOTTOM` wrong is what produces dead motion at the bottom of the throw: the output sits pinned at its endpoint until the raw count crosses it.

Eye joystick

Both axes read centre-zero, **-128 at one extreme, 0 at spring-centre, +128 at the other**, with a deadzone (`EYE_DEADZONE`) so neutral is exactly 0/0.

The deadzone alone is not enough to hold neutral at zero. `stickyBand()` trails its input by up to `band` , so it will settle one count off centre and stay there. `eyeAxis()` therefore forces a hard 0 whenever the mapped value lands in the deadzone rather than letting the band drag toward it -- the same thing `processPotCentred()` does for the centre-zero pots. Without that snap the stick rests at -1 / +1 instead of 0 / 0.

The stick's electrical centre is not the midpoint of its travel, and the two halves of each axis are not the same width, so **each half is mapped on its own scale** from the measured calibration points (`EYE_X_AT_MINUS` / `_AT_ZERO` / `_AT_PLUS` and the Y equivalents). A single straight map across the whole axis would leave neutral off-zero and make one direction hit its endpoint before the other.

Neutral now transmits 128, which is true servo centre. It previously transmitted the stick's own electrical centre (135), leaving the eyes slightly off-centre at rest.

Spike rejection

`median3()` guards every control that drives motion (both neck axes, both eye axes, both faders) by taking the median of the last three **raw** samples.

A dirty wiper momentarily reads somewhere else entirely, and a single bad sample is enough to fling a servo or stepper across its travel. A median discards any single-sample outlier completely, however far out it lands, while a genuine move passes straight through. An averaging or slew-limiting filter would instead smear every fast move in order to soften the rare bad one.

The cost is exactly one sample of lag (40ms at the 25 Hz control rate), and a spike has to repeat on two consecutive samples to get through. Face pots are deliberately not filtered this way: a stray frame on an eyebrow is cosmetic, where the same frame on the neck is the robot lurching.

Sticky band: stillness at rest without losing resolution

`stickyBand()` is the anti-jitter filter used by the neck joystick axes, the eye axes, and fader_left.

The band has to be sized per control, against that control's own sensitivity. A narrow window turns the same ADC noise into a bigger swing in output counts: fader_left gets about 1.6 raw counts per output count against 5.3 on a neck axis, so roughly 8 counts of raw noise is +/-1.5 output counts on the neck but +/-5 on the fader. fader_left therefore has its own `FADER_L_STICKY_BAND` (7) rather than sharing `POT_STICKY_BAND` (3). Simulated at that noise level the fader stops moving at rest from a band of 5 upward, and 7 leaves margin while costing 0.22% of travel in lag.

The obvious filter -- "only report a new value once it has moved N counts, then jump to it" -- does stop jitter, but it also quantises real movement into N-sized steps, so inching a control makes the output hop N at a time instead of counting. That is unusable on an axis you want to move slowly and precisely.

Instead the reported value **trails** the live one by up to `band` and is dragged along rather than snapped:

- **At rest**: noise inside +/- `band` never moves the reported value at all.
- **Moving**: the report follows the input one count at a time, staying `band` behind, so full 1-count resolution survives.

The only cost is `band` counts of lag, under a tenth of a percent of a 3200-count axis.

The **neck joystick axes are filtered at the two axis reads, before the mixer**. `neck_left` and `neck_right` are a differential mix of both axes, so a couple of counts of noise on each reaches the output as a couple of counts of physical stepper motion. Filtering the mixer outputs instead would leave each axis free to jitter into the other. This matters most with the joystick springs removed, when the stick can sit parked off-centre indefinitely.

Simulated against +/-8 raw counts of ADC noise with the stick parked off-centre: `neck_left` changes value **3990 times in 5000 reads unfiltered, and 0 times filtered**. A slow deliberate sweep still resolves 110 distinct values across a 111-count span, so the precision is intact.

Wire formats are unchanged

Every one of these is a controller-side representation change. The ESP-NOW packet still carries eyes as 0-255 and `nL` / `nR` / `nP` as 0-3200 absolute, rescaled at transmit, so **j4_receiver and j4_stepper_neck need no reflash**. j4_stepper_neck turns each of those into an absolute position (halved, homed off that axis's MIN limit switch), so signed values on the wire would command negative positions and run the axes into their limit switches.

The display packet's eye and neck slots are `uint8_t` and get the same re-centring; casting a signed value straight in would wrap it to the top of the range.

Dual-source arbitration

Only the iris is arbitrated now: the IRIS pot on j4_display_right against fader_right, last-mover-wins (`dualPick()`). Whichever moved last beyond `DUAL_CLAIM_COUNTS` is the active source, so the two never fight, and a source that is absent can neither claim nor hold active status.

The neck pivot no longer has a pair -- the NECK PIV pot is disabled and fader_left is its only source -- and Nose Basket is single-source as well.

Values shown on j4_display_right

Every 40ms this board sends j4_display_right the whole effective face as `V:<iris>,<color>,<brightness>,<brow L>,<brow R>,<basket brow L>,<basket brow R>,<nose>,<nose basket>,<eyelid L>,<eyelid R>` (0-255 each, the `FACES.TXT` order). These are the values after arbitration and after any recalled face, so the screen shows what the servos are actually being sent. A channel whose module is absent, with no recalled face holding it, goes out as `-1` and shows as `--`. This replaced the iris-only `I:` line in v0_6_41.

Remote, streamed from j4_display_right's dedicated ADS1115 (raw counts over Serial2, scaled here with the same `processPot()`):

Channel	Control	Destination
A0	Iris	270-deg iris servo (PCA9685 on j4_receiver)
A1	Color	WS2812B strip color (j4_receiver)
A2	Brightness	WS2812B strip brightness (j4_receiver)
A3	Volume	j4_talk audio volume

eyes_x, eyes_y, and iris drive servos on the receiver's PCA9685. The neck joystick mixes into neck-L / neck-R as before. color and brightness drive the receiver's WS2812B strip.

Toggle switches

Four panel toggles, each wired between its GPIO and GND (internal pull-up, ON = LOW). Their states travel in the control packet as a bitmask:

GPIO	Toggle	Bit	Action on the robot
32	LASER	0	Laser servo, PCA9685 ch 14 on j4_receiver
33	VENT	1	Vent-fin servo, PCA9685 ch 15 on j4_receiver
13	EYE POP	2	Eye-pop steppers: sends 0 (normal) or 3200 (popped) down the stepper chain (j4_stepper_neck to j4_stepper_eyes)
15	AUX	3	Next to the right joystick; read and transmitted, not yet assigned

Keypad wiring

Two 4x4 keypads, each on its own PCF8574 backpack on the shared I2C bus:

- **keypad_left** (the newer keypad, left of the panel) at **0x21** -- set the backpack's A0 jumper high. If that backpack turns out to be a PCF8574**A**, the same jumper lands at 0x39; change the two `0x21`s in `main.cpp`.
- **keypad_right** (the original keypad) at **0x20** -- all jumpers low.

The left keypad drives phrase select; the right keypad drives face presets (tap = recall, 3s hold = save). Each can be absent or hot-plugged without affecting the other.

Each keypad is plugged straight into its PCF8574 P0-P7 (keypad pin 1 into P0, in order). The I2CKeyPad library's fixed scan pattern does not match this wiring, so the firmware uses a custom scan via the PCF8574 library directly.

The two pads are different models and **do not put their rows and columns on the same pins**, so both the keymap string and the drive/read pin split are per-keypad (`KeypadPins`, carried in `KeypadGuard`).

Right keypad (original) -- rows on P0, P1, P2, P7; scanner drives those and reads P3, P4, P5, P6:

PCF pin	Keypad wire
P0	Column 3 (3, 6, 9, #)
P1	Column 2 (2, 5, 8, 0)
P2	Column 1 (1, 4, 7, *)

P3	Row 4 (*, 0, #, D)
P4	Row 3 (7, 8, 9, C)
P5	Row 2 (4, 5, 6, B)
P6	Row 1 (1, 2, 3, A)
P7	Column 4 (A, B, C, D)

Left keypad (newer model) -- straight 4+4 split; scanner drives P0-P3 and reads P4-P7:

PCF pin	Keypad wire
P0	Column 1 (1, 4, 7, *)
P1	Column 2 (2, 5, 8, 0)
P2	Column 3 (3, 6, 9, #)
P3	Column 4 (A, B, C, D)
P4	Row 1 (1, 2, 3, A)
P5	Row 2 (4, 5, 6, B)
P6	Row 3 (7, 8, 9, C)
P7	Row 4 (*, 0, #, D)

Why the split matters (bench note, 2026-08-29)

A matrix key is only visible to the scanner if one of its two lines is in the drive set and the other is in the read set. Running the left pad with the right pad's split left six keys (A, B, C, *, 0, #) with both of their lines in the same set, so they produced no scan code at all and the remaining ten came out transposed. No keymap string can recover keys that never generate a code, which is why this needed the per-pad `KeypadPins` change rather than just a re-derived `keymap_left`.

To re-derive a keymap for a replacement pad: press each key, note the character that appears, and rearrange that pad's string so index `drive*4 + read` holds the right character. If some keys produce nothing at all, the drive/read split is wrong, not the string.

Face presets

A **face** is a snapshot of the robot's facial expression: iris, eye color, eye brightness, the eight middle face pots, and the LASER / VENT / EYE POP toggle states. Volume, the neck values, and eyes X/Y are deliberately not part of a face.

On the **right keypad**:

- **Tap a key**: recall that key's saved face. It fires about 40-60ms after the key is let go. Every face channel jumps to the saved value and holds there; turning any face pot takes just that channel back (frozen-baseline takeover, so even a slow turn works), and flipping a toggle takes that toggle back. The rest of the face stays put. A key with nothing saved shows "NO FACE ON `<key>`".
- **Hold an empty key for 3 seconds** (any key, `*` and `#` included): the current face is saved to that key at once, and j4_display_right's header strip shows "FACE SAVED TO `<key>`" for at least 3 seconds, with the SD card result underneath ("SD CARD OK", or "SD WRITE FAILED, KEPT IN CONTROLLER").

- **Hold a key that already has a face for 3 seconds**: the header asks "OVERRIDE FACE `<key>` ?" / "PRESS * TO CONFIRM, OTHER KEY CANCELS" and waits, with no timeout. Press `*` to overwrite the face with the current one; press any other key to cancel ("FACE `<key>` NOT CHANGED"). The key that answers the prompt does nothing else.

Active face

The key last recalled or saved is the **active face**. While no message is up, j4_display_right's header shows it in place of KEVCO, as **Face: A**. As soon as anything differs from what is saved on that key it becomes **Face: A***, and each value that differs gets its own `*`: the face pot numbers in the upper area, and the IRIS / COLOR / BRIGHTNESS labels at the bottom. A flipped LASER / VENT / EYE POP toggle marks the header only, since the toggles have no spot on that screen. Turning a pot back to within 2 counts of its saved value clears its `*`. Saving makes the saved values the held ones, exactly as a recall would, so pot jitter cannot mark a fresh save as changed; a pot has to really move (`DUAL_CLAIM_COUNTS`) to take its channel back.

The active key and a changed-value mask ride on the end of the `V:` line (see j4_display_right). The active face is not remembered across a power cycle; the header says KEVCO until the first recall or save.

Keypad timing

The right pad is scanned every 20ms (`RK_SCAN_MS`), and a key counts as released after 60ms with nothing seen (`RK_RELEASE_MS`). A tap recalls on release, and the recall goes out on the very next loop pass rather than waiting for the 40ms control tick. If the same key reappears within 250ms of a release (`RK_REJOIN_MS`), it is treated as a contact dropout, not a new press: the 3s hold carries on from where it was, and a recall that dropout fired is undone, so a flicker while holding a key to override it cannot wipe the unsaved changes you are about to save. Both keypads and all four ADS1115s also ride out up to two missed I2C probes (`I2C_MISS_LIMIT`) before being declared absent, holding their last values meanwhile.

Where faces are stored

The authoritative store is the **microSD on j4_display_right's TFT module**, in `FACES.TXT`. This board keeps a **cache in its own NVS flash partition** (16 slots at 24 bytes = 384 bytes against the default table's 20KB `nvs` partition).

The split matters:

- **The card is the source of truth.** This board asks for a dump at boot and writes through on every save, over the Serial2 link that already carries `V:` / `M:` / `X:`. A dump overwrites the local cache, the one exception being a locally-dirty slot, which is newer than the card by definition and is queued to be pushed to it.
- **The NVS cache means nothing waits on the card.** A save is in flash before the confirmation is drawn, boot never blocks on a dump, and recall works with j4_display_right unplugged, which is most of bench testing.

Writes happen only on a save, on a mirror acknowledgement, and once on migration, so flash wear is a non-issue even before NVS's own wear levelling. The stored blob carries a version byte plus `FACE_SLOTS` and `FACE_VALUES`, so changing either is detected on load and treated as "nothing stored" rather than reinterpreted as garbage.

Normal firmware uploads leave NVS alone. `pio run -t erase` and changes to the partition table both wipe it.

Sync behaviour

- This board re-requests the dump every 2.5s until a complete one lands (every 10s while the answer is "no card", since each request makes j4_display_right try to mount the slot). A dump only counts as complete when the `FACE:` lines received match the count in `FACE_END:<n>`, so a partial read is simply re-requested.
- `FACE_END:-1` means there is no card, as distinct from `FACE_END:0` for a mounted card with no faces yet. On `-1` this board stays unsynced and keeps asking, so a card inserted later loads without a reboot.
- A save is cached in NVS first, then written through. While j4_display_right is offline the save stays flagged dirty and one dirty face is pushed per 2s tick once the link returns. Only the answer to the save just made is shown;

background syncs stay quiet.
- After a complete dump, every face held here that the card lacks is queued and written to it, so a freshly formatted card fills itself from the faces already in this controller.
- On `FACEERR:` nothing is lost: the face is in NVS regardless. Auto-retry stops for that slot and the dump is re-requested; the next complete dump (the card is back) clears the failure and re-queues it.

The old j4_talk path

The `FACES.TXT` path on j4_talk's microSD is gated off behind `FACE_STORE_TALK_SD`, which defaults to `0`. Two stores both serving dumps would fight, with whichever answered last silently winning, so while the display_right card is in use the controller neither requests from talk nor acts on face packets it sends. The ESP-NOW plumbing is left in place, so setting that define back to `1` restores the old behaviour without touching j4_receiver or j4_talk.

Slots are keyed by the keypad **character**, so re-deriving a keymap keeps every saved face on the same printed key. All 16 keys are usable as slots. Face traffic is its own ESP-NOW packet type (0x04) translated by j4_receiver to text lines on the Teensy UART; see those repos for the protocol.

Dependencies

Managed via PlatformIO (`platformio.ini`):

Library	Version
RobTillaart/PCF8574	0.3.9
RobTillaart/ADS1X15	0.3.13
TFT_eSPI	bundled in `lib/` (pre-configured for TTGO T-Display v1.1)

Built-in ESP-IDF components used: `esp_now`, `WiFi`.

Building and uploading

```
# Build
pio run

# Upload
pio run --target upload

# Monitor serial output
pio device monitor
```

Upload speed is 921600 baud. Serial monitor is 115200 baud. Both are set in `platformio.ini`.

The TFT_eSPI copy in `lib/TFT_eSPI/` is pre-configured for the TTGO T-Display v1.1. Do not replace it with the registry version without reconfiguring.

Receiver

The robot receiver's MAC address is set in `broadcastAddress[]` near the top of `main.cpp`. Update this if the receiver board is swapped.

Related projects

- **j4_display_left** -- the XIAO ESP32S3 jukebox display (portrait, left of the panel)
- **j4_display_right** -- the XIAO ESP32S3 pot-label display (landscape, right of the panel) that streams the iris/color/brightness/volume pots to this board
- **j4_receiver** -- the robot-side TTGO that receives this controller's ESP-NOW packets and returns status